Square Integration

1. Overview

1.1 Description

Square Integration connects **Square** (appointment scheduling and POS platform) with **Newo Platform**.

It enables:

- Checking real-time appointment availability from Square Bookings
- Booking appointments with automatic customer creation in Square
- Cancelling existing appointments
- Synchronizing service catalogs (services, variations, pricing) from Square
- LLM-powered service matching from natural language requests
- Team member assignment based on availability per slot
- Two authentication methods: **Personal Access Token** (recommended) and **OAuth 2.0**
- Auto-configuring conversation canvas with booking and cancellation scenarios

This integration is designed for **service businesses** (salons, spas, healthcare, etc.) who need **automated appointment scheduling through a conversational AI agent** (voice or chat).

1.2 Use Cases

- When a client asks about availability → Query Square Bookings for open slots with matched service
- When a client requests a specific service → LLM matches natural language to service catalog variations
- When a client wants to book → Find or create customer, assign team member, create booking
- When a client wants to cancel → Cancel booking via Square API with idempotency
- When a client requests two or more services at once → Transfer to manager

1.3 Supported Features

Feature	Supported	Notes
Check Availability	✅	By date, time, and service type via Square Bookings API
Book Appointment	✅	With auto customer creation and team member assignment
Cancel Appointment	✅	By booking ID with idempotency key
Service Catalog Sync	✅	Auto-sync on session start, filters APPOINTMENTS_SERVICE
LLM Service Matching	✅	Gemini matches natural language → service variation ID
Team Member Assignment	✅	Auto-selects available team member per slot
Multi-Location Support	✅	Location selection via enum attribute
Idempotent Operations	✅	UUID keys for booking, customer creation, cancellation

Canvas Auto-Setup	✅	Booking + cancellation scenarios added to canvas on publish
Multi-Service → Manager	✅	2+ services requested → transfer to manager
Personal Access Token Auth	✅	No OAuth flow needed — paste token from Developer Dashboard
OAuth 2.0 Auth	✅	Standard OAuth flow with auto token refresh
Reschedule	❌	Cancel + rebook as workaround
Payment Processing	❌	Not implemented

2. Requirements

Before installation:

- Active **Square** account with Bookings enabled
- At least one **location** configured in Square
- Services defined in Square **Catalog** (product type: APPOINTMENTS_SERVICE)
- **One of:**
  - **Personal Access Token** from Square Developer Dashboard (recommended, simpler)
  - **Square OAuth App** with Client ID and Secret

3. How to Setup

3.1 Option A — Personal Access Token (Recommended)

The simplest way to connect. No OAuth flow required.

1. Go to **https://developer.squareup.com/apps**
2. Create a new application (or use an existing one)
3. Go to **Credentials** → **Production** tab
4. Copy the **Application ID** and **Personal Access Token**
5. In Newo platform, set:
   - `square_auth_type` = **Token**
   - `square_personal_token` = your copied token
6. Click **Publish All**

> *Important: after the first successful auth, you must explicitly select `square_locations` and publish again. The integration filters services and availability by the chosen location, so without it the agent will not find Square appointment services.*

> *For a detailed guide with screenshots, see the Square Onboarding Guide (PDF).*

3.2 Option B — OAuth 2.0

Use this if you need scoped access or are connecting on behalf of another seller.

1. In Newo platform, set `square_auth_type` = **OAuth**
2. Set `square_client_id` and `square_secret_id`
3. Click the Square authorization link provided by the platform
4. Log in to your Square account and authorize the app

5. Copy the **authorization code** from the redirect page
6. Paste the code into the `square_oauth_code` attribute
7. Click **Publish All**

> *Important: after the first successful auth, you must explicitly select `square_locations` and publish again. The integration filters services and availability by the chosen location, so without it the agent will not find Square appointment services.*

OAuth scopes requested: `ITEMS_READ`, `APPOINTMENTS_READ`, `APPOINTMENTS_WRITE`, `APPOINTMENTS_ALL_READ`, `APPOINTMENTS_ALL_WRITE`, `EMPLOYEES_READ`, `CUSTOMERS_WRITE`, `CUSTOMERS_READ`, `MERCHANT_PROFILE_READ`

3.3 Attributes Reference

Attribute	Required	Default	Description
`square_auth_type`	✅	`Token`	Authentication method: `Token` or `OAuth`
`square_personal_token`	Token mode	—	Personal Access Token from Square Developer Dashboard
`square_client_id`	OAuth mode	—	OAuth2 Application ID
`square_secret_id`	OAuth mode	—	OAuth2 Application Secret
`square_oauth_code`	OAuth mode	—	One-time OAuth authorization code
`square_locations`	✅	—	Selected location (auto-populated after auth and must be chosen before booking/availability will work)
`square_base_url`	❌	`https://connect.squareup.com`	API base URL
`square_login_url`	❌	`https://app.squareup.com`	Login URL (OAuth only)
`square_enable_slot_check`	❌	`True`	Enable availability checking
`square_enable_booking`	❌	`True`	Enable booking capability
`square_enable_cancellation`	❌	`True`	Enable cancellation capability

`square_setup_scenarios`	❌	`True`	Auto-add booking/cancellation scenarios to canvas
`square_override_agent_attributes`	❌	`True`	Override SuperAgent attributes on publish

4. How to Use

4.1 How the Integration Works

The integration uses Square **Bookings API** for scheduling, **Catalog API** for services, and **Customers API** for contact management. Availability is computed via the bookings availability search, and services are matched from the synced catalog using LLM.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Square

Available Triggers

Trigger	Event	Description
Check Availability	`get_available_slots`	When the agent needs to look up open slots
Book Appointment	`book_slot`	When the agent confirms a booking
Cancel Appointment	`convo_agent_cancel_booking`	When the agent processes a cancellation
Catalog Sync	`session_started`	Auto-sync service catalog on conversation start
Setup	`project_publish_finish`	Initializes integration on deploy
Canvas Setup	`gm_workflow_builder_canvas_built`	Auto-adds booking/cancellation scenarios

Available Actions

- **Search Availability** — Find open slots by service and date range (POST `/v2/bookings/availability/search`)
- **Search Customer** — Find customer by phone (POST `/v2/customers/search`)
- **Create Customer** — New customer record (POST `/v2/customers`)
- **Create Booking** — Schedule appointment with team member (POST `/v2/bookings`)
- **Cancel Booking** — Cancel appointment (POST `/v2/bookings/{id}/cancel`)
- **Search Catalog** — Sync service variations (POST `/v2/catalog/search`)
- **Get Locations** — List merchant locations (GET `/v2/locations`)

4.2 Flows

Flow	Purpose
SetupFlow	Initialize integration: auth (Token or OAuth), connectors, locations

CanvasSetupFlow	Auto-add booking + cancellation scenarios and intents to canvas
UtilsFlow	Execute HTTP requests, handle token refresh (OAuth), error routing
AvailabilityFlow	Check available slots from Square Bookings API
BookingFlow	Create bookings with customer lookup/creation and team member assignment
CancellationFlow	Cancel bookings by booking ID
SetupCatalogFlow	Sync service catalog from Square on session start
MetadataFlow	Version tracking

4.3 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Square Integration
    participant API as Square API

    Note over I,API: Setup (on publish)
    A->>I: project_publish_finish
    alt Token mode
        I->>I: Use Personal Access Token directly
    else OAuth mode
        I->>API: POST /oauth2/token (code)
        API-->>I: access_token + refresh_token
    end
    I->>API: GET /v2/locations
    API-->>I: Location list (populate enum)

    Note over I,API: Canvas Setup (on publish)
    I->>I: gm_workflow_builder_canvas_built
    I->>I: Add booking + cancellation scenarios to canvas

    Note over I,API: Catalog Sync (on session start)
    A->>I: session_started
    I->>API: POST /v2/catalog/search (ITEM, APPOINTMENTS_SERVICE)
    API-->>I: Service variations
    I->>I: Store in square_catalog

    Note over U,API: Availability Check
    U->>A: "Any openings for a manicure tomorrow?"
    A->>I: get_available_slots (date, time, service_request)
    I->>I: LLM match "manicure" -> service_variation_id
    I->>API: POST /v2/bookings/availability/search
    API-->>I: Available slots + team members
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."
```

```
    Note over U,API: Booking (single service)
    U->>A: "Book 2:00 PM please"
    A->>I: book_slot (bookingDateTime)
    I->>API: POST /v2/customers/search (phone)
    alt Customer not found
        I->>API: POST /v2/customers (name, phone)
        API-->>I: customer_id
    end
    I->>API: POST /v2/bookings (location, customer, service, team_member)
    API-->>I: booking_id
    I->>A: result_success (booking_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,A: Multi-service request
    U->>A: "Book manicure and lip filler"
    A->>U: "For multiple services, let me connect you with our manager"
    A->>A: Transfer to manager

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id)
    I->>API: POST /v2/bookings/(id)/cancel
    I->>A: result_success
    A->>U: "Your appointment has been cancelled"
```

AvailabilityFlow

```
flowchart TD
    E["get_available_slots"] --> GATE{{"Slot check<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| LLM["LLM match:<br>service_request →<br>service_variation_id<br>
(temp=0.2)"]
    LLM --> UTC["Convert date/time<br>to UTC range<br>(+ 5 days window)"]
    UTC --> API["POST /v2/bookings<br>/availability/search<br>{location, service,
<br>time range}"]
    API --> OK{{"Success?"}}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| PARSE["Parse slots:<br>extract team_member_id<br>per time slot"]
    PARSE --> TZ["Convert to local<br>timezone (AM/PM)"]
    TZ --> STORE["Store team member<br>map in persona:<br>square_free_teamembers"]
    STORE --> OUT["result_success<br>{availability[]}"]
```

BookingFlow

```
flowchart TD
    E["book_slot"] --> GATE{{"Booking<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| PHONE{{"Phone<br>available?"}}
    PHONE -->|"No"| ASK["urgent_message:<br>ask for phone"]
```

```
    PHONE -->|"Yes"| SEARCH["POST /v2/customers<br>/search {phone}"]
    SEARCH --> FOUND{{"Customer<br>found?"}}
    FOUND -->|"No"| CREATE["POST /v2/customers<br>{name, phone}<br>(idempotency
key)"]
    CREATE --> CID["customer_id"]
    FOUND -->|"Yes"| CID
    CID --> TEAM["Look up team_member_id<br>from square_free_teamembers<br>for
selected time"]
    TEAM --> UTC["Convert time to UTC"]
    UTC --> BOOK["POST /v2/bookings<br>{location, customer,<br>service_variation,
<br>team_member, start_at}<br>(idempotency key)"]
    BOOK --> OUT["result_success<br>{booking_id}"]
```

CancellationFlow

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{{"Cancellation<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| ID["Extract booking_id<br>from payload.meta"]
    ID --> API["POST /v2/bookings<br>/{booking_id}/cancel<br>{idempotency_key}"]
    API --> OK{{"Success?"}}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| OUT["result_success"]
```

SetupCatalogFlow

```
flowchart TD
    E["session_started"] --> TOKEN{{"Access token<br>exists?"}}
    TOKEN -->|"No"| SKIP["Skip"]
    TOKEN -->|"Yes"| API["POST /v2/catalog/search<br>{ITEM type,<br>not deleted}"]
    API --> FILTER["Filter items:<br>product_type ==<br>APPOINTMENTS_SERVICE"]
    FILTER --> LOC["Check location:<br>present_at_all_locations<br>OR in
square_locations"]
    LOC --> EXTRACT["Extract variations:<br>id, name, description,<br>service_name"]
    EXTRACT --> STORE["Store as<br>square_catalog JSON"]
```

UtilsFlow — Token Refresh (OAuth mode only)

```
flowchart TD
    REQ["HTTP Request"] --> RESP{{"Status?"}}
    RESP -->|"200"| BODY{{"Body has<br>error?"}}
    BODY -->|"No"| SUCCESS["square_result_success"]
    BODY -->|"Yes"| ERROR["square_result_error"]
    RESP -->|"401"| AUTH{{"Auth type?"}}
    AUTH -->|"Token"| FAIL["square_result_error<br>(no refresh available)"]
    AUTH -->|"OAuth"| REFRESH["POST /oauth/v2/refresh<br>{client_id,
refresh_token}"]
```

```
    REFRESH --> SAVE["Save new tokens"]
    SAVE --> RETRY["Retry original request"]
```

4.4 Canvas Scenarios

When `square_setup_scenarios` is `True` (default), the CanvasSetupFlow automatically adds these to the conversation canvas on publish:

Scenarios:

- **"Scheduling Appointment via Agent"** (`square_booking_scenario`) — full booking flow with multi-service guard (2+ services → transfer to manager)
- **"Canceling Appointment via Agent"** (`cancellation_via_agent_scenario`) — guided cancellation with confirmation

Intent Types:

- **"[T] Appointment Request via Agent"** (`square_booking_intent`)
- **"[T] Cancellation via Agent"** (`cancellation_via_agent_intent`)

These scenarios can be customized on the canvas after initial setup — modified scenarios are not overwritten on subsequent publishes.

4.5 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice). Each flow has dedicated webhook test endpoints for connectivity validation.

4.6 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify token is set (Token mode) or OAuth exchange completed
2. **Catalog**: Start a conversation — verify `square_catalog` is populated with service variations
3. **Availability**: Ask "Any openings for [service] tomorrow?" — verify slots returned with team members
4. **Booking (single service)**: Complete a booking — verify appointment created in Square
5. **Booking (multi-service)**: Request two services — verify agent transfers to manager
6. **Cancellation**: Cancel a booking — verify appointment deleted from Square

If no action occurs:

- Ensure authentication is configured:
  - **Token mode**: `square_auth_type` = `Token` and `square_personal_token` is set
  - **OAuth mode**: `square_client_id` , `square_secret_id` are set and `square_access_token` was obtained
- Confirm `square_locations` is selected (should be auto-populated as enum)
- Check `square_catalog` is not empty (services must exist in Square as APPOINTMENTS_SERVICE)
- Verify feature flags are enabled (`square_enable_slot_check` , `square_enable_booking` , `square_enable_cancellation`)
- Re-publish the project if credentials were changed

> *Note: This integration creates real appointments in Square. Bookings and cancellations are reflected in the live Square system.*

5. FAQ

Q: What is the difference between Token and OAuth authentication? A: **Token** (Personal Access Token) is simpler — the business owner copies a token from their Square Developer Dashboard. It gives full access to their account without scopes. Best for single-merchant setups. **OAuth** uses the standard authorization flow with scoped permissions. Best for multi-merchant or marketplace scenarios.

Q: How does the agent match services from natural language? A: The integration uses Gemini 2.5 Flash (temperature=0.2) to match the user's service request (e.g., "manicure") against the synced `square_catalog` of service variations. It returns the matching `service_variation_id` used for the availability search and booking.

Q: What happens when a client requests two or more services? A: The agent detects multiple services and does NOT attempt to book. Instead, it explains that multi-service appointments require manager assistance and initiates a transfer (during working hours) or relays a message to the manager (after hours).

Q: How are team members assigned to appointments? A: During the availability search, Square returns which team members are available for each slot. The integration stores this mapping (`square_free_teamembers`) in the persona. When booking, the correct team member is looked up for the selected time.

Q: Why are idempotency keys used? A: All write operations (customer creation, booking, cancellation) use UUID idempotency keys to prevent duplicate operations if requests are retried. This is a Square API best practice.

Q: When is the service catalog synced? A: The catalog syncs automatically on `session_started` (every new conversation). It filters for items with `product_type: APPOINTMENTS_SERVICE` that are available at the selected location.

Q: Can I connect multiple Square locations? A: The integration supports one location per instance. During setup, available locations are fetched and presented as an enum (`square_locations`) in the format `"id | name | timezone"` . Select the desired location after initial auth.

Q: What happens if the token expires or is invalid? A: **Token mode**: The agent returns an error (Personal Access Tokens don't expire, but can be revoked). The business owner needs to generate a new token. **OAuth mode**: The integration detects HTTP 401 responses and automatically refreshes the token via `/oauth/v2/refresh` . Failed requests are retried.

Q: What customer information is needed for booking? A: The customer's phone number is required for lookup/creation. Name (first/last) is used when creating a new customer. The integration splits `full_name` into first and last names automatically.

Q: Can I disable the auto-added canvas scenarios? A: Yes, set `square_setup_scenarios` to `False` before publishing. Existing scenarios on the canvas will not be removed.